

UNICESUMAR – UNIVERSIDADE CESUMAR
ENGENHARIA DE SOFTWARE

FERNANDO QUEIROZ

EXPRESSÕES, ATRIBUIÇÃO E CONTROLE

Relatório da Aula 07 – Paradigmas de Linguagens de Programação

Relatório apresentado à disciplina de Paradigmas
de Linguagens de Programação, do curso de
Engenharia de Software.

Professor: Munif

Maringá – PR

2026

SUMÁRIO

1 INTRODUÇÃO	3
2 EXPRESSÕES	3
2.1 Precedência e associatividade	3
2.2 Ordem de avaliação dos operandos e efeitos colaterais	4
2.3 Sobrecarga de operadores	5
3 CONVERSÕES DE TIPOS	6
3.1 Alargamento, estreitamento e coerção	6
3.2 Coerção implícita em JavaScript e PHP	6
4 EXPRESSÕES RELACIONAIS, BOOLEANAS E CURTO-CIRCUITO	7
4.1 Valores booleanos e comparações	7
4.2 Avaliação em curto-circuito	8
5 SENTENÇAS DE ATRIBUIÇÃO	8
5.1 A atribuição como expressão	8
5.2 Atribuição de modo misto	9
6 ESTRUTURAS DE CONTROLE	9
6.1 Quantas estruturas são necessárias	9
6.2 Seleção de dois caminhos e o else pendente	9
6.3 Seleção múltipla	10
6.4 Laços de repetição	11
6.5 Desvio incondicional: o goto	11
6.6 Comandos protegidos de Dijkstra	12
7 EXERCÍCIOS DE FIXAÇÃO	12
8 CONSIDERAÇÕES FINAIS	13
REFERÊNCIAS	14

1 INTRODUÇÃO

Este relatório apresenta uma síntese organizada do conteúdo da Aula 07 da disciplina de Paradigmas de Linguagens de Programação, do curso de Engenharia de Software, cujo tema é Expressões, Atribuição e Controle. O conteúdo tem como referência os capítulos 7 e 8 do livro de Sebesta (2018), páginas 299 a 361, e foi trabalhado em uma aula exploratória, na qual a turma prevê a saída de cada trecho de código antes de executá-lo.

O objetivo da aula é compreender como as linguagens de programação avaliam expressões e controlam o fluxo de execução, de modo que o código faça exatamente o que aparenta fazer, sem surpresas de precedência, de conversão de tipos ou de desvio. Para o engenheiro de software, isso se traduz em três competências práticas: ler código de terceiros sem ser enganado pela endentação ou pela precedência; evitar conversões implícitas perigosas, como as do operador `==` em JavaScript e PHP; e escolher a estrutura de controle mais legível para cada situação.

A relevância do tema é demonstrada por falhas reais. Em 2014, uma linha `goto fail;` duplicada no código da Apple desviava a execução antes da verificação de assinatura digital, e iOS e macOS passaram a aceitar certificados falsos. Em 2008, no último dia de um ano bissexto, um laço `while` dos tocadores Zune, da Microsoft, nunca terminava, e todos os aparelhos travaram. Em PHP, o operador `==` considera iguais strings como `"0e4..."` e `"0e8..."`, o que fez com que logins que comparavam hashes com esse operador aceitassem senhas incorretas.

O relatório segue a sequência da aula: expressões (precedência, efeitos colaterais e sobrecarga), conversões de tipos, expressões booleanas e curto-circuito, atribuição, estruturas de controle e, por fim, os exercícios de fixação e a síntese. Os trechos de código são apresentados por meio de capturas dos slides originais, identificadas como figuras.

2 EXPRESSÕES

2.1 Precedência e associatividade

Ao projetar a avaliação de expressões aritméticas, uma linguagem precisa responder a seis questões: quais são as regras de precedência; quais são as regras

de associatividade; em que ordem os operandos são avaliados; se há restrições a efeitos colaterais; se o usuário pode sobrecarregar operadores; e que mistura de tipos é permitida (SEBESTA, 2018, p. 300-305). A expressão $a + b * c$, com $a = 3$, $b = 4$ e $c = 5$, mostra o impacto das duas primeiras: avaliada da esquerda para a direita, resulta em 35; avaliada da direita para a esquerda, resulta em 23; e, com a precedência usual de $*$ sobre $+$, também resulta em 23. A linguagem APL avalia sempre da direita para a esquerda, e Lisp dispensa a precedência por usar notação prefixa, em que os parênteses determinam a ordem, como em $(+ a (* b c))$.

A Figura 1 compara expressões equivalentes em Python, JavaScript e Java. Em Python, $-2 ** 2$ resulta em -4 e $2 ** 3 ** 2$ resulta em 512, pois o operador $**$ associa à direita. Em JavaScript, $-2 ** 2$ é um erro de sintaxe, o que obriga o programador a explicitar a intenção com parênteses. A expressão $1 + 2 + "3"$ produz `TypeError` em Python e `"33"` em JavaScript e Java. A divisão e o resto com operandos negativos também variam: $-7 / 2$ resulta em -4 (com `//`) em Python, -3,5 em JavaScript e -3 em Java, e $-7 \% 2$ resulta em 1 em Python e em -1 nas outras duas linguagens.

Figura 1 – Precedência e associatividade em Python, JavaScript e Java

Precedência e associatividade na prática

Expressão	Python	JavaScript	Java
$-2 ** 2$	-4	erro de sintaxe	(não há $**$)
$2 ** 3 ** 2$	512	512	-
$1 + 2 + "3"$	<code>TypeError</code>	<code>"33"</code>	<code>"33"</code>
$-7 / 2$ (inteiros)	-4 (com <code>//</code>)	-3,5	-3
$-7 \% 2$	1	-1	-1

Associatividade (p. 303)
 Esquerda: $*$ / $\%$ + -
 Direita: $**$ (Python, JS, Ruby, Fortran),
 ++, --, menos unário
 Visual Basic: $^$ à esquerda!

Excel
 $=-2^2$ dá 4; lá o menos unário vence o $^$.

Na dúvida, **use parênteses** (p. 305): eles deixam a intenção explícita para o compilador e para o próximo leitor.

Sebesta, 11ª ed. - §7.2.1.1-7.2.1.3 - p. 301-305 - exemplos/01_precedencia

Fonte: Aula 07 (slide 6).

Os operadores $*$, $/$, $\%$, $+$ e $-$ associam à esquerda, enquanto $**$ (em Python, JavaScript, Ruby e Fortran), $++$, $--$ e o menos unário associam à direita. Visual Basic é uma exceção, pois seu $^$ associa à esquerda. No Excel, $=-2^2$ resulta em 4, porque ali o menos unário tem precedência sobre o $^$. Em caso de dúvida,

recomenda-se o uso de parênteses, que tornam a intenção explícita tanto para o compilador quanto para o próximo leitor (SEBESTA, 2018, p. 305).

2.2 Ordem de avaliação dos operandos e efeitos colaterais

Há efeito colateral quando uma função altera uma variável externa ao seu escopo local. No exemplo de Sebesta (2018, p. 307), a variável global `a` vale 5, e a função `fun1` a altera para 17 e retorna 3. Na expressão `a = a + fun1()`, o resultado depende de qual operando é avaliado primeiro: se `a` for lido antes da chamada, o resultado é 8; se for lido depois, é 20. Em Java, a ordem é garantida, da esquerda para a direita, e o resultado é 8. Em C, o padrão não define a ordem, e o gcc 14 produziu 20. Em Go, a especificação também não define a leitura de `a` em relação à chamada (Figura 2).

Figura 2 – Ordem dos operandos e efeitos colaterais

Ordem dos operandos e efeitos colaterais

Exemplo do livro (p. 307)

```
int a = 5;

int fun1(void) {
    a = 17;      /* efeito colateral */
    return 3;
}

int main(void) {
    a = a + fun1(); /* 8 ou 20? */
    printf("a = %d\n", a);
}
```

8

Java

a ordem é garantida: da esquerda para a direita (p. 308)

20

C (gcc 14)

a ordem não é definida pelo padrão

20

Go

a especificação não define a leitura de `a` em relação à chamada

Transparência referencial (p. 308): sem efeitos colaterais, a ordem deixa de importar, e $f(x) + f(x)$ pode virar $2 * f(x)$.

Sebesta, 11ª ed. - §7.2.2 - p. 307-309 - exemplos/02_efeitos_colaterais

Fonte: Aula 07 (slide 7).

Quando não há efeitos colaterais, a ordem de avaliação deixa de importar, propriedade conhecida como transparência referencial (SEBESTA, 2018, p. 308). Nesse cenário, $f(x) + f(x)$ pode ser substituída por $2 * f(x)$ com segurança. Daí a recomendação de preferir funções puras e de evitar expressões cujo resultado dependa de uma ordem de avaliação não especificada.

2.3 Sobrecarga de operadores

Um operador é sobrecarregado quando o mesmo símbolo assume significados diferentes conforme o contexto. O `+` soma inteiros, soma números reais e, em Java, também concatena strings. A sobrecarga pode prejudicar a legibilidade:

em C++, o símbolo `&` representa tanto o E bit a bit quanto o operador "endereço de" (SEBESTA, 2018, p. 309). A sobrecarga definida pelo usuário é permitida em C++, C#, Python, Rust e Kotlin, mas não em Java, que recorre a métodos nomeados, como `x.add(y).multiply(tres)` na classe `BigDecimal`.

Em Python, a sobrecarga é feita por métodos especiais: uma classe `Vetor` que define `__add__` e `__mul__` passa a aceitar `v1 + v2` e `v * 3`, de modo que a soma de `Vetor(1, 2)` com `Vetor(3, 4)` resulta em `Vetor(4, 6)`, e multiplicá-lo por 3 resulta em `Vetor(12, 18)`. Em Java, `'a' + 1` resulta em 98, enquanto `"" + 'a' + 1` resulta em `a1`, pois a presença da string muda o significado do `+`.

3 CONVERSÕES DE TIPOS

3.1 Alargamento, estreitamento e coerção

Uma conversão de alargamento transforma um valor em um tipo capaz de representar todos os valores do tipo de origem; a de estreitamento faz o contrário. Quando a linguagem converte automaticamente, fala-se em coerção; quando o programador a solicita, em conversão explícita ou `cast` (SEBESTA, 2018, p. 310-313). O alargamento costuma ser seguro, mas pode perder precisão: em Java, ao atribuir `123456789` a um `float` e converter o resultado de volta para `int`, obtém-se `123456792`. O estreitamento exige `cast` e pode descartar informação: `(int) 3.99` resulta em `3` e `(byte) 200` resulta em `-56`. Em Java, `byte s = b + c`; não compila mesmo com `b` e `c` do tipo `byte`, porque a soma é promovida a `int`. Por fim, `1 / 2` resulta em `0`, enquanto `1 / 2.0` resulta em `0,5`.

Figura 3 – Alargamento, estreitamento e coerção

Conversões de tipos: alargamento, estreitamento e coerção

Java - coerção e cast

```
int grande = 123456789;
float f = grande;          // alargamento
                           // implícito
System.out.println((int) f); // 123456792 !

double d = 3.99;
System.out.println((int) d); // 3 (cast)
System.out.println((byte) 200); // -56

byte b = 10, c = 20;
// byte s = b + c; // erro: b + c é int (p. 312)

System.out.println(1 / 2); // 0
System.out.println(1 / 2.0); // 0.5
```

Linguagem	int → real (implícito)	int32 + int64
C	sim	coerção
Java	sim (alargamento)	coerção para long
Python	sim	(um só tipo int)
Go	não: float64(i)	erro de compilação
Rust	não: i as f64	erro de compilação
ML, F# (p. 312)	não	erro de compilação

Coerção esconde erros (p. 312): em `d = b * a;`, com `a` digitado por engano no lugar de `c`, o Java converte e compila.

Sebesta, 11ª ed. - §7.4 - p. 310-313 - exemplos/04_conversoes

Fonte: Aula 07 (slide 9).

O principal risco da coerção é que ela esconde erros. Em `d = b * a;`, se `a` foi digitado por engano no lugar de `c`, Java converte o valor e compila sem aviso (SEBESTA, 2018, p. 312). Por esse motivo, Go, Rust, ML e F# exigem conversão explícita, como `float64(i)` em Go e `i as f64` em Rust, e tratam a soma de `int32` com `int64` como erro de compilação, ao passo que C e Java promovem o operando menor por coerção.

3.2 Coerção implícita em JavaScript e PHP

JavaScript e PHP são exemplos de linguagens com coerção agressiva (Figura 4). Em JavaScript, `"5" - 2` resulta em 3, pois o operando textual é convertido em número, mas `"5" + 2` resulta em `"52"`, pois o `+` com uma string significa concatenação. Também produzem valores surpreendentes `true + 1` (2), `[] + {}` (`"[object Object]"`), `null + 1` (1) e `undefined + 1` (NaN), e a expressão `"b" + "a" + +"a" + "a"` resulta em `"baNaN"`. Entre os operadores de igualdade, `0 == ""` e `0 == "0"` são verdadeiros, mas `"" == "0"` é falso, de modo que `0 ==` não é transitivo. O operador `===`, que não faz coerção, retorna falso para `0 === "0"`.

Figura 4 – Coerção implícita em JavaScript e PHP

Coerção implícita: JavaScript e PHP

JavaScript	PHP - "type juggling"
<pre> "5" - 2 // 3 "5" + 2 // "52" "5" * "2" // 10 true + 1 // 2 [] + {} // "[object Object]" null + 1 // 1 undefined + 1 // NaN "b" + "a" + "a" + "a" // "baNaNa" 0 == "" // true "" == "0" // false 0 == "0" // true (não é transitivo!) 0 === "0" // false (=== não converte) </pre>	<pre> var_dump("0e123" == "0e456"); // true ! var_dump("0e123" === "0e456"); // false var_dump("1" == "01"); // true md5("240610708") // "0e462097431906..." md5("QNKCDZO") // "0e830400451993..." // == entre os dois hashes: true </pre>
Regra prática Em JavaScript e PHP, compare com === e !==, que não fazem coerção.	

Sebesta, 11ª ed. - §7.4.1 Coerção em expressões - p. 311-312 - exemplos/04_conversoes

Fonte: Aula 07 (slide 10).

Em PHP, o chamado "type juggling" faz "0e123" == "0e456" ser verdadeiro, pois ambas as strings são interpretadas como números em notação científica, iguais a zero. Os hashes MD5 de "240610708" e de "QNKCDZO" começam com 0e seguido apenas de dígitos, de modo que a comparação com == os considera iguais, o que originou o caso de login citado na introdução. A regra prática, nas duas linguagens, é comparar sempre com === e !==.

4 EXPRESSÕES RELACIONAIS, BOOLEANAS E CURTO-CIRCUITO

4.1 Valores booleanos e comparações

Em C, o resultado de uma comparação é um inteiro, 0 ou 1, e qualquer valor diferente de zero é tratado como verdadeiro (SEBESTA, 2018, p. 315). Por isso $3 > 2 > 1$ resulta em 0: a primeira comparação, $3 > 2$, vale 1, e $1 > 1$ é falso. Em Python, as comparações encadeadas têm o sentido matemático, e $a > b > c$ equivale a $(a > b) \text{ and } (b > c)$, resultando em `True` (Figura 5).

Figura 5 – Expressões relacionais e booleanas

Expressões relacionais e booleanas

C - booleano é inteiro (p. 315)

```
int a = 3, b = 2, c = 1;
printf("%d\n", a > b > c); // 0 !
// (3 > 2) vale 1, e 1 > 1 é falso

printf("%d\n", (a > b) && (b > c)); // 1

if (-1) printf("verdadeiro\n"); // != 0
```

Python - comparações encadeadas

```
a, b, c = 3, 2, 1
print(a > b > c) # True: (a > b) and (b > c)

# contam como FALSO em um if:
# 0 0.0 "" [] {} None
# contam como VERDADEIRO:
# "0" [0] " "
```

Linguagem	O que o if aceita
C, Python, JavaScript	qualquer valor (0, vazio e nulo contam como falso)
Java, C#, Go, Rust	somente booleano: if (n) com n inteiro nem compila

Sebesta, 11ª ed. - §7.5 - p. 313-315 - exemplos/05_booleanas

Fonte: Aula 07 (slide 11).

Em C, JavaScript e Python, o `if` aceita qualquer valor. Em Python, contam como falsos `0`, `0.0`, `""`, `[]`, `{}` e `None`, e contam como verdadeiros `"0"`, `[0]` e `" "`. Já Java, C#, Go e Rust aceitam somente valores booleanos, e `if (n)` com `n` inteiro nem compila, o que elimina uma classe inteira de erros.

4.2 Avaliação em curto-circuito

Na avaliação em curto-circuito, o segundo operando de `&&` ou de `||` só é avaliado se o primeiro não determinar o resultado (SEBESTA, 2018, p. 315-317). Essa propriedade permite escrever a busca do livro, `while (i < lista.length && lista[i] != chave)`, sem acessar `lista[lista.length]`. Em Java, `s != null && s.length() > 0` retorna falso quando `s` é nulo, enquanto `s != null & s.length() > 0`, com o operador `&`, que não faz curto-circuito, lança `NullPointerException`.

Em JavaScript, o encadeamento opcional `?.` retorna `undefined` quando a referência é nula, e o operador `??` substitui apenas `null` e `undefined`, ao contrário de `||`, que também descarta o valor `0`: com `qtd = 0`, `qtd || 10` resulta em `10`, e `qtd ?? 10` resulta em `0`. É preciso cuidado com efeitos colaterais no segundo operando: em `(a > b) || (b++ / 3)`, o incremento só ocorre quando `a <= b`. Em Python, `and` e `or` devolvem um dos operandos, e não um booleano, de modo que `0 or "padrão"` resulta em `"padrão"`.

5 SENTENÇAS DE ATRIBUIÇÃO

5.1 A atribuição como expressão

Em C, C++ e JavaScript, a atribuição é uma expressão e produz um valor. Isso permite atribuições com múltiplos alvos, como `sum = count = 0`, e idiomas como `while ((ch = getchar()) != EOF)`, mas também permite o erro clássico `if (x = y)`, que compila, atribui o valor de `y` a `x` e testa o resultado (SEBESTA, 2018, p. 319-320). Java e C# também tratam a atribuição como expressão, porém exigem booleano no `if`, o que impede o erro.

Python não trata a atribuição como expressão, exceto pelo operador morsa `:=`, introduzido no Python 3.8. Em Go, nem `x++` é expressão, e em Rust a atribuição vale `()`, de modo que `if x = 5` não compila. Python oferece ainda atribuição múltipla, que permite trocar valores sem variável temporária (`a, b = b, a`). Nas linguagens funcionais, `val` (ML) e `let` (F# e Rust) apenas dão nome a um valor que não muda (SEBESTA, 2018, p. 321).

5.2 Atribuição de modo misto

Java só admite alargamento implícito: `double d = 3;` é válido, `int i = 3.7;` é erro de compilação e `int i = (int) 3.7;` produz 3. Há, porém, uma exceção sutil: `i += 3.7` compila, pois o operador composto embute um cast, e, com `i = 3`, o resultado é 6. Em C, a coerção ocorre em qualquer direção: `int i = 3.7;` resulta em 3 sem aviso, `char c = 300;` resulta em 44 com apenas um aviso e `unsigned int u = -1;` resulta em 4294967295 (SEBESTA, 2018, p. 321-322).

Uma armadilha clássica é `double taxa = acertos / total * 100;` com operandos inteiros: a divisão inteira ocorre antes do alargamento para `double`, e o resultado é 0.0.

6 ESTRUTURAS DE CONTROLE

6.1 Quantas estruturas são necessárias

Böhm e Jacopini (1966) demonstraram que sequência, seleção e laço lógico bastam para expressar qualquer algoritmo, o que torna o `goto` desnecessário. As linguagens oferecem estruturas adicionais, como `for`, `switch`, `do` e `foreach`, por

facilidade de escrita. O limite é a legibilidade: quanto maior a linguagem, menor a parcela do código alheio que o leitor reconhece (SEBESTA, 2018, p. 328-330).

6.2 Seleção de dois caminhos e o else pendente

No problema do else pendente, a endentação sugere uma associação que a linguagem ignora. No exemplo em C da Figura 6, o `else` está alinhado ao primeiro `if`, mas pertence ao mais interno; como `sum == 0` é verdadeiro e `count == 0` é falso, o resultado é 1 (SEBESTA, 2018, p. 331).

Figura 6 – O else pendente e seletores que devolvem valor

Seleção de dois caminhos: o "else pendente"

C - a endentação engana (p. 331)

```
int sum = 0, count = 5, result = -1;
if (sum == 0)
    if (count == 0)
        result = 0;
else
    // de qual if é este
    result = 1;
printf("%d\n", result); // 1
```

Regra de Java, C, C++ e C#

O `else` pertence ao `if` mais próximo que ainda não tem `else`. A endentação é ignorada (exceto em Python e F#).

Chaves obrigatórias eliminam o problema: Perl, Go e Rust (p. 332).

Seletores que devolvem valor

```
let y = if x > 0 { x } else { 2 * x }; // Rust: if é uma expressão (como em F#,
p. 334)
y = x if x > 0 else 2 * x # Python: expressão condicional (p. 306)
```

Sebesta, 11ª ed. - 58.2.1 - p. 330-334 - exemplos/07_selecao

Fonte: Aula 07 (slide 16).

Em Java, C, C++ e C#, o `else` pertence ao `if` mais próximo que ainda não tem `else`, e a endentação é ignorada; apenas Python e F# a levam em conta. Perl, Go e Rust eliminam o problema ao exigir chaves. Em Rust e F#, o `if` é uma expressão que devolve valor (`let y = if x > 0 { x } else { 2 * x };`), e Python oferece a expressão condicional `y = x if x > 0 else 2 * x`.

6.3 Seleção múltipla

No `switch` de C, a ausência de `break` faz o controle continuar no caso seguinte. No exemplo da Figura 7, com `index = 1`, são executados o incremento de `odd`, o de `even` e ainda a mensagem de erro do `default` (SEBESTA, 2018, p. 335).

Figura 7 – Do switch de C ao match

Seleção múltipla: do switch de C ao match

C - sem break, o controle "cai" (p. 335)

```
switch (index) {
  case 1:
  case 3: odd += 1;
  case 2:
  case 4: even += 1;
  default: printf("Error\n");
}
// index = 1: odd, even E erro!
```

Java 14+ - switch como expressão

```
String tipo = switch (dia) {
  case SAB, DOM -> "fim de
semana";
  case SEG, TER, QUA,
    QUI, SEX -> "dia útil";
}; // sem break, sem cair;
// exaustivo para enum
```

Rust - match exaustivo

```
match n {
  i32::MIN..=-1 => "negativo",
  0 => "zero",
  1..=9 => "um dígito",
  _ => "vários dígitos",
}
// sem o _ : não compila
```

Go: cada caso termina sozinho, e cair exige `fallthrough`. C#: cair implicitamente é proibido (p. 336). Python 3.10: `match` com padrões e guardas.

Sebesta, 11ª ed. - §8.2.2 - p. 334-341 - exemplos/07_selecao

Fonte: Aula 07 (slide 17).

Java 14 introduziu o `switch` como expressão, com `->`, sem queda e exaustivo para enumerações. Rust usa `match` exaustivo: sem o caso `_`, o código não compila. Em Go, cada caso termina sozinho e a queda exige `fallthrough`; em C#, cair implicitamente é proibido; e Python 3.10 adicionou `match` com padrões e guardas.

6.4 Laços de repetição

No `for` de C, a contagem é controlada por expressões arbitrárias, e o laço pode alterar o próprio contador, de modo que o exemplo da aula imprime 0 3 6 9; trata-se de um laço lógico disfarçado. Em Python, o `for` percorre um iterável, e alterar `i` no corpo não tem efeito: o laço imprime de 0 a 9. As questões de projeto envolvem o tipo e o escopo da variável de controle, a possibilidade de alterá-la no corpo e a avaliação dos parâmetros uma só vez ou a cada volta (SEBESTA, 2018, p. 342).

Figura 8 – Laços lógicos, saídas e iteradores

Laços lógicos, saídas e iteradores

Java - break rotulado (p. 348)

```
externo:
for (int i = 0; i < m.length; i++)
  for (int j = 0; j < m[i].length; j++)
    if (m[i][j] < 0) {
      System.out.println(i + ", " + j);
      break externo; // sai dos DOIS
    }
laços
}
```

JavaScript - for...in × for...of

```
const notas = [10, 20, 30];
let total = 0;
for (const i in notas) total += i;
console.log(total); // "0012": chaves!

total = 0;
for (const v of notas) total += v;
console.log(total); // 60: valores
```

Zune (2008) - o laço lógico sem progresso

```
while (dias > 365) {
  if (bissexto(ano)) {
    if (dias > 366) { dias -= 366; ano++; }
    // dias == 366: nada muda, laço infinito!
  } else { dias -= 365; ano++; }
}
```

Pós-teste (do-while) executa **ao menos uma vez**, e isso é fácil de esquecer (p. 347).

Sebesta, 11ª ed. - §8.3.2-8.3.4 - p. 345-352 - exemplos/08_iteracao

Fonte: Aula 07 (slide 19).

Entre os mecanismos de saída, o `break` rotulado de Java encerra laços aninhados de uma só vez. Em JavaScript, `for...in` percorre as chaves e `for...of` percorre os valores; com `notas = [10, 20, 30]`, o primeiro produz a string "0012" e o segundo, 60. O laço de pós-teste (do-while) executa ao menos uma vez, detalhe fácil de esquecer. O caso do Zune ilustra um laço lógico sem progresso: quando restam exatamente 366 dias em um ano bissexto, nenhuma das ramificações altera `dias` ou `ano`, e o laço nunca termina.

6.5 Desvio incondicional: o goto

O `goto` é o desvio incondicional. Na versão simplificada do caso Apple (Figura 9), o segundo `goto fail;` não pertence ao `if` e, portanto, desvia sempre: a verificação da assinatura nunca é alcançada, e a função retorna `err` igual a 0, que significa certificado válido.

Figura 9 – O defeito "goto fail" da Apple (2014)

Desvio incondicional: o goto

"goto fail" (Apple, 2014) - versão simplificada

```
int valida_certificado(void) {
    int err;
    if ((err = verifica_hash()) != 0)
        goto fail;
    goto fail;           // duplicada: desvia
SEMPRE
    if ((err = verifica_assinatura()) != 0)
        goto fail;       // nunca é alcançada
fail:
    return err;           // 0 = "certificado
válido"
}
```

Dijkstra (1968)

"Go To Statement Considered Harmful"

Quem tem goto hoje

C, C++, C#, Go. Java não tem (p. 353)

Gotos restritos

break, continue e return melhoram a legibilidade (p. 353)

Com chaves obrigatórias (Go, Rust) ou com gcc -Wall (aviso de endentação enganosa), o bug teria sido pego.

Sebesta, 11ª ed. - §8.4 - p. 352-353 - exemplos/09_desvio_protegido

Fonte: Aula 07 (slide 20).

Dijkstra (1968) publicou o artigo "Go To Statement Considered Harmful". Hoje possuem `goto` as linguagens C, C++, C# e Go, enquanto Java não o tem. Desvios restritos, como `break`, `continue` e `return`, melhoram a legibilidade (SEBESTA, 2018, p. 352-353). Chaves obrigatórias, como em Go e Rust, ou o aviso de endentação enganosa do `gcc -Wall` teriam evitado o defeito.

6.6 Comandos protegidos de Dijkstra

Os comandos protegidos associam uma guarda a cada ramo (SEBESTA, 2018, p. 353-355). Todas as guardas são avaliadas; se várias forem verdadeiras, a escolha é não determinística; se nenhuma for, ocorre erro no `if` ou o laço termina no `do ... od`. No exemplo do livro, `if x >= y -> max := x [] y >= x -> max := y fi`, quando `x == y` qualquer ramo serve. O conceito é a base do CSP de Hoare (1978) e do `select` de Go, que sorteia entre os canais prontos: em mil repetições, o exemplo da aula resultou em uma distribuição próxima de 505 para `a` e 495 para `b`, variando a cada execução.

7 EXERCÍCIOS DE FIXAÇÃO

A aula foi aberta com um quiz de previsão de saída (Quadro 1) e encerrada com o exercício em duplas "encontre o bug" (Quadro 2), no qual o slide apresenta apenas os trechos e pede, para cada um, a saída, a regra responsável pelo problema e a correção. As respostas abaixo foram elaboradas a partir do conteúdo estudado.

Quadro 1 – Gabarito do quiz de abertura

#	Código	Saída
1	Python: <code>print(-2 ** 2, 2 ** 3 ** 2)</code>	-4 512
2	Java: <code>1 + 2 + "3" + 4 + 5</code>	3345
3	JavaScript: <code>"5" - 2, "5" + 2</code>	3 52
4	C: <code>printf("%d\n", 3 > 2 > 1)</code>	0
5	Python: <code>x = 0; print(x or "padrão", 3 > 2 > 1)</code>	padrão True
6	JavaScript: <code>0 == "", "" == "0", 0 == "0"</code>	true false true

Fonte: elaborado pelo autor, com base na Aula 07 (slide 4).

Quadro 2 – Resolução do exercício "encontre o bug"

#	Linguagem	O que acontece	Correção
1	JavaScript	<code>for...in</code> itera chaves: imprime "Total: 0012"	Usar <code>for...of</code>
2	Python	<code>d or 10</code> trata 0 como falso: <code>desconto(200, 0)</code> dá 180.0, e não 200.0	<code>if d is None: d = 10</code>
3	Java	<code>switch</code> sem <code>break</code> cai até o <code>default</code> : imprime "inválido"	<code>break</code> em cada caso
4	C	O ; após o <code>if</code> é um comando vazio: as duas mensagens são sempre impressas	Remover o ;
5	Go	<code>:=</code> cria um novo <code>x</code> no bloco (shadowing): dentro: 20 e fora: 10	<code>x = x * 2</code> , se a intenção era alterar o <code>x</code> externo
6	Java	<code>7 / 10</code> é divisão inteira: imprime 0.0%	<code>100.0 * acertos / total</code>

Fonte: elaborado pelo autor, com base na Aula 07 (slides 22 a 24).

8 CONSIDERAÇÕES FINAIS

O conjunto de tópicos estudados mostra que cada armadilha de expressões e de controle tem uma contrapartida em mecanismos adotados pelas próprias linguagens. A Figura 10 resume essa relação: a precedência é controlada com parênteses; a ordem dos operandos, com regras fixas, como em Java, ou com funções puras; as conversões, com conversão explícita (Go e Rust) e com `===`; os booleanos, com `if` restrito a valores booleanos; a atribuição, ao deixar de ser expressão; a seleção, com `match` exaustivo e chaves obrigatórias; os laços, com `for...of`, iteradores e `break` rotulado; e o desvio, com gotos restritos.

Figura 10 – Síntese: armadilhas e como as linguagens ajudam**Síntese: armadilhas e como as linguagens ajudam**

Tema	Armadilha	Como as linguagens ajudam
Precedência	<code>-2 ** 2, 1 + 2 + *3</code>	parênteses; JS obriga <code>(-2) ** 2</code>
Operandos	<code>a + f()</code> com efeito colateral (C, Go)	Java fixa a ordem; funções puras
Conversões	coerção silenciosa (C, JS, PHP)	Go e Rust exigem conversão explícita; <code>==</code>
Booleanos	<code>0, "", []</code> são falsos; <code>a > b > c</code> em C	<code>if</code> só com booleano (Java, Go, Rust); <code>??</code>
Atribuição	<code>if (x = y)</code>	atribuição não é expressão (Python, Go, Rust)
Seleção	<code>switch</code> sem <code>break</code> ; <code>else</code> pendente	<code>-></code> , <code>match</code> exaustivo, chaves obrigatórias
Laços	<code>for...in</code> no JS; laço sem progresso	<code>for...of</code> , iteradores, <code>break</code> rotulado
Desvio	<code>goto</code> sem restrição	<code>break</code> , <code>continue</code> , <code>return</code> : <code>gotos</code> restritos

Sebesta, 11ª ed. - Caps. 7 e 8 - p. 299-361

Fonte: Aula 07 (slide 25).

Como atividades complementares, a aula propõe estudar expressões em Ruby e em Lisp, a implementação da seleção múltipla por tabelas de desvio, os laços em linguagens funcionais e os iteradores de Ruby, além da leitura do artigo de Dijkstra (1968) e de uma comparação do `switch` entre C, Java, Go e Rust quanto à queda entre casos, à exaustividade e à legibilidade. A próxima aula trata de subprogramas (capítulos 9 e 10).

REFERÊNCIAS

SEBESTA, Robert W. **Conceitos de linguagens de programação**. 11. ed. Porto Alegre: Bookman, 2018.